AFW Session 4 - Build Guide

Student Query Escalation & Contextual Reply | Apps Script + Supabase + Telegram

Flow: PENDING Sheet row -> Supabase lookup -> Telegram if human review is needed -> Gmail reply -> PROCESSED

Classroom simulation only. Use fictional student records. Do not use real confidential student data or share credentials.

What you are building

Session 3 already receives the email, classifies it with Gemini, checks approved policy in NotebookLM, and writes cases needing more work to the AFW Query Escalation Log as PENDING. Session 4 starts from that PENDING row.

- Apps Script reads the waiting case from Google Sheets.
- Supabase returns the fictional student record using the sender email address.
- Telegram alerts a human reviewer only when Gemini marked the query as needing human review.
- The workflow combines policy + student record + review status into a contextual reply.
- Gmail replies to the original message and the Sheet closes the case as PROCESSED.

The one-paste rule

You paste three files only once: (1) the Supabase SQL, (2) Code.gs, and (3) appsscript.json. The **only edit** you make is the email address in three places - two in the SQL and one in Code.gs. After that, you never type or delete code: you select a function and press Run.

Before you start

- Use your **FIIB Google account** for everything: Workspace Studio, the Sheet, Apps Script and Gmail.
- Install Telegram on your phone (or open web.telegram.org) and sign in.
- Pair up with a partner who will send test emails to your FIIB address. Note down the email they send from.
- Keep this guide open - the three code files are in Appendices A, B and C.

Sheet structure - know the queue before you code

Col	Header	Filled by	Purpose
A	Received At	Session 3	Email date and time
B	Student Email	Session 3	Sender email address - used for the Supabase lookup
C	Subject	Session 3	Email subject - shown in the Telegram alert
D	Email Body	Session 3	Email body
E	Gemini Analysis	Session 3	Gemini classifier output - checked for NEEDS_HUMAN_REVIEW: YES
F	Processing Status	Both	PENDING -> PROCESSED (or REPLY FAILED)
G	Supabase Record	Session 4	Student record returned by Supabase
H	Telegram Status	Session 4	ALERT SENT or NOT REQUIRED
I	Policy Answer	Session 3	NotebookLM output - used in the reply
J	Email ID	Session 3	Gmail message ID - used to reply in the same thread
K	Reply Status	Session 4	REPLY SENT or REPLY FAILED: <reason>

Session 3 fills A-F, I and J. Session 4 fills G, H and K, and changes F from PENDING to PROCESSED.

Build sequence

#	Run / Action	What should happen
1	Supabase SQL - Blocks A to E	Table, demo student, lookup function and test all work in Supabase.
2	testReadPendingRows	Apps Script can see the Session 3 PENDING row.
3	testSupabaseLookup	Apps Script calls Supabase: log shows 200 and the student JSON.
4	getTelegramChatId	Your Telegram chat ID appears in the execution log.
5	sendTelegramTest	Telegram receives the connection test message.
6	processLatestPendingRow	One function does the whole case: G filled -> H set -> Telegram if needed -> Gmail reply -> K = REPLY SENT -> F = PROCESSED.
7	Time-driven trigger	processLatestPendingRow runs every minute without pressing Run.

Phase 1 - Check that Session 3 works

1. Open Workspace Studio and go to **AFW Student Query Assistant**.
2. Make sure the flow is **turned on**.
3. Ask your partner to send this email to your FIIB address:
   - Subject: AFW attendance status
   - Body: I missed one AFW session. Does this affect my attendance or assessment eligibility?
4. Wait about a minute.
5. Open the Google Sheet **AFW Query Escalation Log** and click the **Sheet1** tab.
6. Find the newest row at the bottom and check:
   - B Student Email = your partner's email. Write it down - you need it in Phases 2 and 3.
   - E Gemini Analysis and I Policy Answer are filled in.
   - J Email ID is filled in.
   - F Processing Status = PENDING. G, H and K are blank.

Stop condition: you have a genuine PENDING row created by the Session 3 flow. Do not type a row by hand - it has no Email ID and the reply step will fail later.

Phase 2 - Build the student database in Supabase

Create your account and project

1. Go to **supabase.com** and click **Start your project**.
2. Click **Continue with Google** and choose your FIIB account.
3. If asked to create an organization: type any name, choose **Personal** and the **Free** plan, and click **Create organization**.
4. Click **New project** and fill in:
   - Project name: AFW Session 4
   - Database password: click Generate a password, and save it somewhere safe
   - Region: Mumbai (or the nearest available)
5. Click **Create new project** and wait 1-2 minutes until the dashboard is ready.

Paste the SQL and update the email

6. In the left sidebar click **SQL Editor**. A blank query opens (if not, click **+ New query**).
7. Copy the whole of **Appendix A** and paste it into the editor.
8. Find the two lines with **<ENTER EMAIL HERE>** (one in the insert, one in the last line). Replace that text with your partner's email from Phase 1. Keep the quote marks.

UPDATE THE EMAIL: replace <ENTER EMAIL HERE> with the sender's email ID - the email that sends the AFW test mail. Example: 'priya.sharma@gmail.com'

Run it one block at a time

Select a block with your mouse, then click **Run** (bottom right). Supabase runs only the part you selected.

9. **Block A** - select from create table down to the first); and Run. You should see: Success. No rows returned.
10. **Block B** - select from insert into down to its); and Run. You should see: Success.
11. **Block C** - select the two lines select * from public.afw_student_records; and Run. Your demo student appears in the results.
12. Click **Table Editor** in the left sidebar, then **afw_student_records**. You can see the same student as a table. Go back to **SQL Editor**.
13. **Block D** - select from create or replace function down to $$; and Run. You should see: Success.
14. **Block E** - select the last line select public.get_afw_student_record(...) and Run. The result is JSON containing "found": true.

Supabase code rule: paste the complete SQL once and keep every section. Never press Run on the whole file a second time - you will get "already exists" errors.

What you have now: give Supabase an email address and it returns that student's record.

Phase 3 - Open Apps Script and paste the code

Apps Script is Google's built-in automation tool. It lives inside your Sheet, can read and write the Sheet, and can call services outside Google - like Supabase and Telegram.
Workspace Studio -> Sheet -> Apps Script -> Supabase / Telegram / Gmail

Open Apps Script

1. Go back to the **AFW Query Escalation Log** Sheet.
2. In the top menu click **Extensions -> Apps Script**. A new tab opens.
3. Click **Untitled project** (top left), rename it **AFW Session 4** and click **Rename**.
4. Look at the screen:
   - Left sidebar icons: Editor < >, Triggers (clock), Project Settings (gear)
   - Files list: Code.gs
   - Toolbar: Save, Run, and the function dropdown
   - Execution log: appears at the bottom after you press Run

Paste Code.gs

5. Click inside **Code.gs**, press **Ctrl+A** (Cmd+A on Mac) and delete the default myFunction.
6. Copy the whole of **Appendix B** and paste it into Code.gs.
7. Press **Ctrl+F**, search for **ENTER EMAIL**, and replace <ENTER EMAIL HERE> with the same partner email you used in Supabase. Keep the quote marks.
8. Click the **Save** icon.

UPDATE THE EMAIL: in testSupabaseLookup, replace <ENTER EMAIL HERE> with the sender's email ID - the same email you entered in Supabase.

Paste appsscript.json

9. Click **Project Settings** (gear) in the left sidebar.
10. Tick **Show "appsscript.json" manifest file in editor**.
11. Click **Editor < >**. appsscript.json now appears in the files list - click it.
12. Press **Ctrl+A**, delete, paste **Appendix C**, and click **Save**.

Add your first Script Property

13. In the Sheet's address bar, copy the part between **/d/** and **/edit** - this is your Sheet ID.
14. In Apps Script, click **Project Settings** and scroll down to **Script Properties**.
15. Click **Add script property**. Property: **SHEET_ID**. Value: paste your Sheet ID.

16. Click **Save script properties**.

Run your first function

17. Click **Editor < >**. In the toolbar's function dropdown choose **testReadPendingRows**.
18. Click **Run**.
19. The first time, Google asks for permission:
    - Click Review permissions and choose your FIIB account
    - If you see "Google hasn't verified this app": click Advanced -> Go to AFW Session 4
    - Click Allow
20. The **Execution log** opens at the bottom and shows your PENDING row from Phase 1.

Do not run anything under the WORKSPACE STUDIO CUSTOM STEP section at the top of Code.gs. That part is blocked by FIIB admin policy and is kept only to show the design.

What you have now: Apps Script can see the case waiting in your Sheet.

Phase 4 - Connect Apps Script to Supabase

Copy the two connection values from Supabase

1. Go back to the Supabase tab and open your **AFW Session 4** project.
2. Click **Project Settings** (gear at the bottom of the left sidebar) -> **Data API**. Copy the **Project URL**. (The **Connect** button at the top also shows it.)
3. Still in Project Settings, click **API Keys**. Copy the **Publishable key** (starts with sb_publishable_). If your screen shows an anon public key instead, use that.

Never copy or use the **Secret key**. The Publishable key is the correct one for this class.

Save them in Apps Script

4. In Apps Script go to **Project Settings -> Script Properties -> Add script property**.
5. Add **SUPABASE_URL** = your Project URL.
6. Add **SUPABASE_KEY** = your Publishable key.
7. Click **Save script properties**.

Test the connection

8. Click **Editor**, choose **testSupabaseLookup** in the dropdown, and click **Run**. Allow permissions if asked.
9. The Execution log shows **200**, then your student's JSON record.

Property	What to paste	Added in	Handling
SHEET_ID	The part of the Google Sheet URL between /d/ and /edit	Phase 3	Not secret
SUPABASE_URL	Supabase Project URL	Phase 4	Configuration
SUPABASE_KEY	Supabase Publishable key (sb_publishable_...)	Phase 4	Never use the Secret key
TELEGRAM_BOT_TOKEN	Token BotFather gives you after /newbot	Phase 5	Secret - do not share or project
TELEGRAM_CHAT_ID	Number shown by getTelegramChatId	Phase 5	Treat as private

What you have now: Google just called an outside database and got a student record back.

Phase 5 - Connect Telegram (the human channel)

Create your bot

1. Open Telegram and search for **BotFather** - pick the one with the blue verified tick.

2. Tap **Start**, then send **/newbot**.
3. When asked for a name, send: **AFW Human Review Bot**
4. When asked for a username, send something ending in **bot**, e.g. afw_review_sk_bot. If it is taken, try another.
5. BotFather replies with your **token** automatically. Copy the long text after "Use this token to access the HTTP API". Do not share it with anyone.

Save the token and find your chat ID

6. In Apps Script add the Script Property **TELEGRAM_BOT_TOKEN** = your token, and save.
7. In BotFather's message, tap the link to your new bot (t.me/...). Tap **Start** and send **hello**.
8. In Apps Script choose **getTelegramChatId** and click **Run**.
9. In the Execution log find **"chat":{"id":** followed by a number. Copy only that number. (If you see "result":[] - send hello to your bot again and re-run.)
10. Add the Script Property **TELEGRAM_CHAT_ID** = that number, and save.

Test it

11. Choose **sendTelegramTest** and click **Run**.
12. Telegram shows: **AFW escalation bot is connected successfully.**

You should now have **five Script Properties**: SHEET_ID, SUPABASE_URL, SUPABASE_KEY, TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID.
Do not share or project your Telegram bot token. It is a credential.

What you have now: Apps Script can send a message to the human reviewer.

Phase 6 - Close the loop with processLatestPendingRow

This one function does the whole Session 4 job. Before running it, press **Ctrl+F**, search for **processLatestPendingRow**, and read its labelled blocks with your Sheet open next to it:

Block in the code	What happens in the Sheet
READ WORKSPACE STUDIO DATA	Reads B, C, E, I and J from the newest PENDING row
SUPABASE LOOKUP	Email in B -> Supabase -> record written to G
HUMAN REVIEW DECISION	Checks E for NEEDS_HUMAN_REVIEW: YES
TELEGRAM ESCALATION	YES -> Telegram alert, H = ALERT SENT. NO -> H = NOT REQUIRED
BUILD STUDENT-FACING RESPONSE	Policy (I) + record (G) + review status -> reply text
SEND STUDENT REPLY	Replies in the original Gmail thread (J). K = REPLY SENT, F = PROCESSED

Case A - the student needs data, not a human

1. Your Phase 1 row should still be **PENDING**.
2. Choose **processLatestPendingRow** and click **Run**. Allow the Gmail permission if asked.
3. Check three places:
   - Sheet: G filled, H = NOT REQUIRED, K = REPLY SENT, F = PROCESSED
   - Gmail: your partner's email now has a reply in the same thread
   - Telegram: nothing arrives - correct, no human was needed

This function performs **real actions**: it sends a real Gmail reply and, when needed, a real Telegram alert.

Case B - the student needs a human decision

4. Ask your partner to send:
   - Subject: AFW deadline exception
   - Body: I missed the AFW WIP submission deadline. Can you allow me to submit it late without penalty?
5. Wait for the new **PENDING** row. Check Column E says **NEEDS_HUMAN_REVIEW: YES**.

6. Run **processLatestPendingRow** again.
7. Check:
   - Telegram: an AFW HUMAN REVIEW REQUIRED alert
   - Sheet: H = ALERT SENT, K = REPLY SENT, F = PROCESSED
   - Gmail: the reply says faculty review is required and nothing was granted automatically

What you have now: one function takes a case from PENDING to PROCESSED.

Phase 7 - Automate it

1. In Apps Script click **Triggers** (clock icon) in the left sidebar.
2. Click **+ Add Trigger** (bottom right) and set:
   - Choose which function to run: processLatestPendingRow
   - Choose which deployment should run: Head
   - Select event source: Time-driven
   - Select type of time based trigger: Minutes timer
   - Select minute interval: Every minute
3. Click **Save** and allow permissions if asked.
4. Ask your partner to send one more AFW email. **Do not click anything.**
5. Within 2-3 minutes the new row goes from PENDING to PROCESSED, the reply arrives in Gmail, and Telegram alerts if needed.

Final architecture: Email -> Gemini -> NotebookLM -> PENDING -> Apps Script -> Supabase -> Telegram if needed -> Gmail reply -> PROCESSED.

Test emails

Case	Subject	Email body	Expected behaviour
A - Student data only	AFW attendance status	I missed one AFW session. Does this affect my attendance or assessment eligibility?	PENDING -> Supabase record in G -> H = NOT REQUIRED -> contextual reply -> PROCESSED
B - Human review	AFW deadline exception	I missed the AFW WIP submission deadline. Can you allow me to submit it late without penalty?	PENDING -> Supabase -> Telegram alert -> H = ALERT SENT -> reply says faculty review required -> PROCESSED
C - Optional policy-only contrast	AFW end-term question	Does AFW have a separate end-term examination?	Decide should normally be FALSE (no student record or human decision needed), so no PENDING row is created.

Governance and classroom rules

- Use fictional classroom student records only. Do not load real marks, attendance, contact details, or sensitive student data into the demo database.
- Use the Supabase Publishable key in the classroom script. Do not use or display a Supabase Secret key.
- Treat TELEGRAM_BOT_TOKEN as a credential. Do not paste it into chat, slides, or shared documents.
- Gemini identifies whether human review is needed; it does not grant exceptions, waivers, reassessments, deadline extensions, or special permission.
- NotebookLM remains the policy layer. Supabase supplies student-specific facts. Telegram is the human-authority path. Apps Script orchestrates the workflow.
- A human-review case receives an automated acknowledgement, but the final decision remains with faculty.
- Supabase marks your table "Unrestricted" because row level security is off. That is fine for fictional data - but a real agent should only reach data through the one lookup function you built.

Quick troubleshooting

Problem	Check
No PENDING row	Check the Workspace Studio flow is ON and the subject contains AFW. Send the test email again. Do not type a row by hand.
testReadPendingRows fails or shows []	Check SHEET_ID (only the part between /d/ and /edit) and that the tab is named Sheet1. A row already PROCESSED is no longer PENDING.
Supabase returns found: false	The email does not match. It must be identical in the SQL insert, the SQL test line and Code.gs (testSupabaseLookup) - and it must be the email that sent the test mail.
"relation already exists" / duplicate key in Supabase	The whole SQL file was run again. Only run the block you need.
Supabase 401 / 403	Re-copy SUPABASE_URL and SUPABASE_KEY (no extra spaces). Use the Publishable key, never the Secret key. If it still fails, run this once in the SQL Editor: grant execute on function public.get_afw_student_record(text) to anon;
Supabase 404 / function not found	Run Block D (create or replace function) and check the name is get_afw_student_record.
getTelegramChatId shows "result":[]	Open your bot, tap Start, send hello, then run getTelegramChatId again.
sendTelegramTest fails	Check TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID for spaces or typos. Make sure you started the bot.
H = NOT REQUIRED on the deadline case	The code looks for the exact text NEEDS_HUMAN_REVIEW: YES in Column E. If Gemini wrote it differently (Yes, bold), resend the email.
K / F show REPLY FAILED	The row has no real Gmail Email ID (typed by hand), or Apps Script is running under a different Google account than the inbox. Use a real email and the FIIB account everywhere.
Trigger does not seem to run	Check Triggers: function = processLatestPendingRow, time-driven, every minute; approve permissions. It handles one PENDING row per run.

Appendix A - Supabase SQL - paste once

Paste the complete script once into SQL Editor. **Update the email in two places** (the red lines): replace <ENTER EMAIL HERE> with the sender's email ID. Then select and run one block at a time (A-E). Never run the whole file twice.

Block	Select from...	Expected result
A	create table ... down to the first);	Success. No rows returned
B	insert into ... down to its);	Success - 1 row inserted
C	select * from public.afw_student_records;	The demo student appears as a row
D	create or replace function ... down to $$;	Success. No rows returned
E	select public.get_afw_student_record(...)	JSON with "found": true

```
create table public.afw_student_records (
  email text primary key,
  student_name text not null,
  program text not null,
  afw_sessions_attended integer not null,
  afw_sessions_missed integer not null,
  pending_assessments integer not null default 0,
  evidence_folder_status text,
  notes text
);

insert into public.afw_student_records (
  email,
  student_name,
  program,
  afw_sessions_attended,
  afw_sessions_missed,
  pending_assessments,
  evidence_folder_status,
  notes
)
values (
  -- UPDATE: Replace <ENTER EMAIL HERE> with the sender's email ID (the email that will send the AFW test mail).
  '<ENTER EMAIL HERE>',
  'Demo Student',
  'PGDM',
  19,
  1,
  0,
  'On track',
  'Fictional classroom demonstration record'
);

select *
from public.afw_student_records;

create or replace function public.get_afw_student_record(p_email text)
returns jsonb
language sql
security definer
set search_path = ''
as $$

  select coalesce(
    (
      select jsonb_build_object(
        'found', true,
        'email', r.email,
        'student_name', r.student_name,
        'program', r.program,
        'afw_sessions_attended', r.afw_sessions_attended,
        'afw_sessions_missed', r.afw_sessions_missed,
        'pending_assessments', r.pending_assessments,
        'evidence_folder_status', r.evidence_folder_status,
        'notes', r.notes
      )
```

```
      from public.afw_student_records r
      where lower(r.email) = lower(trim(p_email))
      limit 1
    ),
    jsonb_build_object(
      'found', false,
      'email', p_email
    )
  );

$$;


-- UPDATE: Replace <ENTER EMAIL HERE> with the same sender's email ID you used in the insert above.
select public.get_afw_student_record('<ENTER EMAIL HERE>');
```

Appendix B - Code.gs - paste once

Delete only the default myFunction(), then paste this complete file once. **Update the email in one place** (the red lines in testSupabaseLookup): replace <ENTER EMAIL HERE> with the same sender's email ID used in Supabase. After that, never edit or delete code - select a function from the dropdown and press Run.

```
/**********************************************
 * AFW EXTERNAL API CONNECTOR
 *
 * Full architecture:
 *
 * Student Email
 *      ↓
 * Workspace Studio
 *      ↓
 * Gemini classification
 *      ↓
 * NotebookLM policy answer
 *      ↓
 * Google Sheet queue
 *      ↓
 * Apps Script
 *      ↓
 * Supabase student lookup
 *      ↓
 * Telegram escalation if human review needed
 *      ↓
 * Reply sent back to student
 **********************************************/


/**********************************************
 * WORKSPACE STUDIO CUSTOM STEP
 *
 * Currently blocked by FIIB Workspace admin policy.
 * Kept here so the clean connector architecture
 * can still be demonstrated in class.
 **********************************************/

function pushCard(card) {
  return {
    "action": {
      "navigations": [
        {
          "push_card": card
        }
      ]
    }
  };
}


function saveButton() {
  return {
    "text": "Save",
    "onClick": {
      "hostAppAction": {
        "workflowAction": {
          "saveWorkflowAction": {}
        }
      }
    }
  };
}


function onConfigStudentLookup() {

  const card = {
    "sections": [
      {
        "header": "AFW Student Lookup",
        "widgets": [
          {
            "textInput": {
              "name": "studentEmail",
```

```
              "label": "Student email",
              "hintText": "Use the sender email from an earlier step",
              "hostAppDataSource": {
                "workflowDataSource": {
                  "includeVariables": true
                }
              }
            }
          },
          {
            "buttonList": {
              "buttons": [
                saveButton()
              ]
            }
          }
        ]
      }
    ]
  };

  return pushCard(card);
}


function outputVariables(variableDataMap) {

  const workflowAction =
    AddOnsResponseService
      .newReturnOutputVariablesAction()
      .setVariableDataMap(variableDataMap);

  const hostAppAction =
    AddOnsResponseService
      .newHostAppAction()
      .setWorkflowAction(workflowAction);

  return AddOnsResponseService
    .newRenderActionBuilder()
    .setHostAppAction(hostAppAction)
    .build();
}


function onExecuteStudentLookup(event) {

  const email =
    event.workflow.actionInvocation.inputs[
      "studentEmail"
    ].stringValues[0];

  const testRecord = JSON.stringify({
    student_email: email,
    connector_status: "WORKING"
  });

  const outputs = {
    "studentRecord":
      AddOnsResponseService
        .newVariableData()
        .addStringValue(testRecord)
  };

  return outputVariables(outputs);
}



/************************************************
 * SUPABASE
 ************************************************/

function testSupabaseLookup() {

  const props =
    PropertiesService.getScriptProperties();

  const url =
    props.getProperty('SUPABASE_URL') +
```

```
    '/rest/v1/rpc/get_afw_student_record';

  const key =
    props.getProperty('SUPABASE_KEY');

  const response =
    UrlFetchApp.fetch(url, {
      method: 'post',
      contentType: 'application/json',
      headers: {
        apikey: key,
        Authorization: 'Bearer ' + key
      },
      payload: JSON.stringify({
        // UPDATE: Replace <ENTER EMAIL HERE> with the sender's email ID
        // (the same email you entered in Supabase).
        p_email: '<ENTER EMAIL HERE>'
      }),
      muteHttpExceptions: true
    });

  Logger.log(
    response.getResponseCode()
  );

  Logger.log(
    response.getContentText()
  );
}


function lookupStudentByEmail(email) {

  const props =
    PropertiesService.getScriptProperties();

  const url =
    props.getProperty('SUPABASE_URL') +
    '/rest/v1/rpc/get_afw_student_record';

  const key =
    props.getProperty('SUPABASE_KEY');

  const response =
    UrlFetchApp.fetch(url, {
      method: 'post',
      contentType: 'application/json',
      headers: {
        apikey: key,
        Authorization: 'Bearer ' + key
      },
      payload: JSON.stringify({
        p_email: email
      }),
      muteHttpExceptions: true
    });

  if (
    response.getResponseCode() !== 200
  ) {

    throw new Error(
      'Supabase lookup failed: ' +
      response.getResponseCode() +
      ' ' +
      response.getContentText()
    );
  }

  return JSON.parse(
    response.getContentText()
  );
}



/**********************************************
 * GOOGLE SHEET QUEUE
 **********************************************/
```

```
function testReadPendingRows() {

  const props =
    PropertiesService.getScriptProperties();

  const sheetId =
    props.getProperty('SHEET_ID');

  const sheet =
    SpreadsheetApp
      .openById(sheetId)
      .getSheetByName('Sheet1');

  const rows =
    sheet
      .getDataRange()
      .getValues();

  const pendingRows =
    rows.filter(
      (row, index) =>
        index > 0 &&
        row[5] === 'PENDING'
    );

  Logger.log(pendingRows);
}



/**********************************************
 * TELEGRAM
 **********************************************/

function getTelegramChatId() {

  const props =
    PropertiesService.getScriptProperties();

  const token =
    props
      .getProperty('TELEGRAM_BOT_TOKEN')
      .trim();

  const url =
    'https://api.telegram.org/bot' +
    token +
    '/getUpdates';

  const response =
    UrlFetchApp.fetch(url, {
      muteHttpExceptions: true
    });

  Logger.log(
    response.getContentText()
  );
}


function sendTelegramTest() {

  const props =
    PropertiesService.getScriptProperties();

  const token =
    props
      .getProperty('TELEGRAM_BOT_TOKEN')
      .trim();

  const chatId =
    props
      .getProperty('TELEGRAM_CHAT_ID')
      .trim();

  const url =
    'https://api.telegram.org/bot' +
    token +
```

```
    '/sendMessage';

  const payload = {
    chat_id: chatId,
    text:
      'AFW escalation bot is connected successfully.'
  };

  const response =
    UrlFetchApp.fetch(url, {
      method: 'post',
      contentType: 'application/json',
      payload: JSON.stringify(payload),
      muteHttpExceptions: true
    });

  Logger.log(
    response.getResponseCode()
  );

  Logger.log(
    response.getContentText()
  );
}


function sendTelegramAlert(message) {

  const props =
    PropertiesService.getScriptProperties();

  const token =
    props
      .getProperty('TELEGRAM_BOT_TOKEN')
      .trim();

  const chatId =
    props
      .getProperty('TELEGRAM_CHAT_ID')
      .trim();

  const url =
    'https://api.telegram.org/bot' +
    token +
    '/sendMessage';

  const payload = {
    chat_id: chatId,
    text: message
  };

  const response =
    UrlFetchApp.fetch(url, {
      method: 'post',
      contentType: 'application/json',
      payload: JSON.stringify(payload),
      muteHttpExceptions: true
    });

  if (
    response.getResponseCode() !== 200
  ) {

    throw new Error(
      'Telegram send failed: ' +
      response.getResponseCode() +
      ' ' +
      response.getContentText()
    );
  }

  return JSON.parse(
    response.getContentText()
  );
}



/**********************************************
```

```
 * STUDENT REPLY
 ***********************************************/

function extractPolicyAnswer(policyText) {

  let text =
    String(policyText || '').trim();

  const match =
    text.match(
      /POLICY_ANSWER:\s*([\s\S]*?)(?=\n\s*STUDENT_RECORD_NEEDED:|\n\s*HUMAN_REVIEW_NEEDED:|\n\s*REASON:|$)/
    );

  if (match) {
    text = match[1].trim();
  }

  /*
   * NotebookLM may include citation markers
   * such as [1], [2], [1, 3].
   * Remove these from the student-facing email.
   */
  text =
    text.replace(
      /\s*\[\d+(?:\s*,\s*\d+)*\]/g,
      ''
    );

  return text.trim();
}


function buildStudentReply(
  policyText,
  studentRecord,
  needsHumanReview
) {

  const policyAnswer =
    extractPolicyAnswer(
      policyText
    );

  let recordText = '';


  if (
    studentRecord &&
    studentRecord.found
  ) {

    recordText =
      '\n\nYour available AFW record currently shows:\n' +

      '- Sessions attended: ' +
      studentRecord.afw_sessions_attended +
      '\n' +

      '- Sessions missed: ' +
      studentRecord.afw_sessions_missed +
      '\n' +

      '- Pending assessments: ' +
      studentRecord.pending_assessments +
      '\n' +

      '- Evidence folder status: ' +
      studentRecord.evidence_folder_status;
  }


  if (needsHumanReview) {

    return (
      'Dear Student,\n\n' +

      policyAnswer +

      recordText +
```

```
      '\n\nYour request requires faculty review and has been escalated. ' +
      'No exception, waiver, approval, reassessment, deadline extension, ' +
      'or other special permission has been granted automatically. ' +
      'You will receive a decision after faculty review.\n\n' +
      'Regards,\n' +
      'AFW Course Team'
    );
  }


  return (
    'Dear Student,\n\n' +
    policyAnswer +
    recordText +
    '\n\nThis response is based on the AFW policy knowledge base ' +
    'and the course record currently available to the system.\n\n' +
    'Regards,\n' +
    'AFW Course Team'
  );
}


function sendStudentReply(
  emailId,
  replyText
) {
  if (!emailId) {
    throw new Error(
      'No Gmail Email ID was provided.'
    );
  }

  const message =
    GmailApp.getMessageById(
      String(emailId).trim()
    );

  if (!message) {
    throw new Error(
      'Could not find Gmail message: ' +
      emailId
    );
  }

  message.reply(
    replyText
  );
}



/************************************************
 * MAIN QUEUE PROCESSOR
 *
 * Sheet columns:
 *
 * A - Received At
 * B - Student Email
 * C - Subject
 * D - Email Body
 * E - Gemini Analysis
 * F - Processing Status
 * G - Supabase Record
 * H - Telegram Status
 * I - Policy Answer
 * J - Email ID
```

```
 * K - Reply Status
 *
 ********************************************/

function processLatestPendingRow() {

  const props =
    PropertiesService.getScriptProperties();

  const sheetId =
    props.getProperty('SHEET_ID');

  const sheet =
    SpreadsheetApp
      .openById(sheetId)
      .getSheetByName('Sheet1');

  const lastRow =
    sheet.getLastRow();


  /*
   * Start with newest row.
   * Process one pending query per trigger run.
   */
  for (
    let row = lastRow;
    row >= 2;
    row--
  ) {

    const status =
      sheet
        .getRange(row, 6)
        .getValue();


    if (
      status !== 'PENDING'
    ) {
      continue;
    }


    /***************************************
     * READ WORKSPACE STUDIO DATA
     ***************************************/

    const email =
      sheet
        .getRange(row, 2)
        .getValue();

    const subject =
      sheet
        .getRange(row, 3)
        .getValue();

    const analysis =
      sheet
        .getRange(row, 5)
        .getValue();

    const policyAnswer =
      sheet
        .getRange(row, 9)
        .getValue();

    const emailId =
      sheet
        .getRange(row, 10)
        .getValue();


    /***************************************
     * SUPABASE LOOKUP
     ***************************************/

    const studentRecord =
```

```
      lookupStudentByEmail(
        email
      );


    // Column G
    sheet
      .getRange(row, 7)
      .setValue(
        JSON.stringify(
          studentRecord
        )
      );


    /**************************************
     * HUMAN REVIEW DECISION
     **************************************/

    const needsHumanReview =
      String(analysis)
        .includes(
          'NEEDS_HUMAN_REVIEW: YES'
        );


    /**************************************
     * TELEGRAM ESCALATION
     **************************************/

    if (needsHumanReview) {

      const existingTelegramStatus =
        sheet
          .getRange(row, 8)
          .getValue();


      /*
       * Prevent duplicate Telegram alerts
       * if the row is retried.
       */
      if (
        existingTelegramStatus !==
        'ALERT SENT'
      ) {

        const telegramMessage =
          '🚨 AFW HUMAN REVIEW REQUIRED\n\n' +

          'Student: ' +
          email +
          '\n' +

          'Subject: ' +
          subject +
          '\n\n' +

          'Gemini Analysis:\n' +
          analysis +
          '\n\n' +

          'Student Record:\n' +
          JSON.stringify(
            studentRecord,
            null,
            2
          );


        sendTelegramAlert(
          telegramMessage
        );


        // Column H
        sheet
          .getRange(row, 8)
          .setValue(
```

```
          'ALERT SENT'
        );
    }

  } else {

    // Column H
    sheet
      .getRange(row, 8)
      .setValue(
        'NOT REQUIRED'
      );
  }


  /****************************************
   * BUILD STUDENT-FACING RESPONSE
   ****************************************/

  const replyText =
    buildStudentReply(
      policyAnswer,
      studentRecord,
      needsHumanReview
    );


  /****************************************
   * SEND STUDENT REPLY
   ****************************************/

  try {

    sendStudentReply(
      emailId,
      replyText
    );


    // Column K
    sheet
      .getRange(row, 11)
      .setValue(
        'REPLY SENT'
      );


    // Column F
    sheet
      .getRange(row, 6)
      .setValue(
        'PROCESSED'
      );


    Logger.log(
      'Processed row ' +
      row +
      ' - reply sent.'
    );

  } catch (error) {

    /*
     * Do not silently lose the failure.
     * Telegram/Supabase work remains visible,
     * and the row records the email error.
     */

    sheet
      .getRange(row, 11)
      .setValue(
        'REPLY FAILED: ' +
        error.message
      );


    sheet
      .getRange(row, 6)
```

```
        .setValue(
          'REPLY FAILED'
        );


      Logger.log(
        'Reply failed for row ' +
        row +
        ': ' +
        error.message
      );

      throw error;
    }


    return;
  }


  Logger.log(
    'No pending rows found.'
  );
}
```

Appendix C - appsscript.json - paste once

Project Settings -> tick Show "appsscript.json" manifest file in editor -> open appsscript.json -> replace everything with this -> Save. The AFW Student Lookup custom step it describes is blocked by FIIB admin policy; it is kept to show the cleaner design and is not run in class.

```
{
  "timeZone": "Asia/Kolkata",
  "exceptionLogging": "STACKDRIVER",
  "runtimeVersion": "V8",
  "addOns": {
    "common": {
      "name": "AFW External API Connector",
      "logoUrl": "https://www.gstatic.com/images/branding/product/1x/googleg_48dp.png",
      "useLocaleFromApp": true
    },
    "flows": {
      "workflowElements": [
        {
          "id": "afwStudentLookup",
          "state": "ACTIVE",
          "name": "AFW Student Lookup",
          "description": "Looks up a student's structured record from Supabase.",
          "workflowAction": {
            "inputs": [
              {
                "id": "studentEmail",
                "description": "Student email address",
                "cardinality": "SINGLE",
                "dataType": {
                  "basicType": "STRING"
                }
              }
            ],
            "outputs": [
              {
                "id": "studentRecord",
                "description": "Student record returned by Supabase",
                "cardinality": "SINGLE",
                "dataType": {
                  "basicType": "STRING"
                }
              }
            ],
            "onConfigFunction": "onConfigStudentLookup",
            "onExecuteFunction": "onExecuteStudentLookup"
          }
        }
```

```
      ]
    }
  }
}
```